

Assignment No-17

Assignment – Circular Singly Linked List

Problem Statement

Implement a **Circular Singly Linked List** in C and perform the following operations:

1. AddFirst
2. AddLast
3. Display
4. DeleteFirst
5. DeleteLast
6. AddPosition
7. DeletePosition

Develop a **menu-driven program** to perform all the above operations.

Concepts Used

- Structures (`struct`)
 - Self-Referential Structures
 - Dynamic Memory Allocation (`malloc()`, `free()`)
 - Pointers
 - Circular Singly Linked List
 - Functions
 - Menu-Driven Programming
-

Data Structure

```
struct Node
{
    int data;
    struct Node *next;
};
```

Functions to Implement

```
void AddFirst(int value);  
void AddLast(int value);  
void Display();  
void DeleteFirst();  
void DeleteLast();  
void AddPosition(int position, int value);  
void DeletePosition(int position);  
int Count();
```

Operations

1. AddFirst(int value)

Insert a new node at the beginning of the circular linked list.

Example

Before

```
20 → 30 → 40  
↑       ↓  
└────────┘
```

After `AddFirst(10)`

```
10 → 20 → 30 → 40  
↑       ↓  
└────────┘
```

2. AddLast(int value)

Insert a new node at the end of the circular linked list.

Example

Before

```
10 → 20 → 30  
↑       ↓  
└────────┘
```

After `AddLast(40)`

```
10 → 20 → 30 → 40
↑           ↓
└──────────┘
```

3. Display()

Display all elements of the circular linked list.

Traverse until the first node is reached again.

Output

```
10 20 30 40
```

4. DeleteFirst()

Delete the first node of the circular linked list.

Handle:

- Empty List
- Single Node
- Multiple Nodes

5. DeleteLast()

Delete the last node of the circular linked list.

Handle:

- Empty List
- Single Node
- Multiple Nodes

6. AddPosition(int position, int value)

Insert a node at the specified position.

Valid Position

```
1 <= Position <= Count + 1
```

Special Cases

- Position = 1 → AddFirst()
- Position = Count + 1 → AddLast()

7. DeletePosition(int position)

Delete the node from the specified position.

Valid Position

```
1 <= Position <= Count
```

Special Cases

- Position = 1 → DeleteFirst()
- Position = Count → DeleteLast()

Algorithm

AddFirst()

1. Create a new node.
2. Store the data.
3. If the list is empty:
 - Make the node point to itself.
 - Update First.
4. Otherwise:
 - Traverse to the last node.
 - Make the new node point to First.
 - Update last node's next.
 - Update First.

AddLast()

1. Create a new node.
2. If the list is empty:
 - Make the node point to itself.
 - Update First.
3. Otherwise:

- Traverse to the last node.
 - Insert the new node after the last node.
 - Point the new node to First.
-

Display()

1. Check whether the list is empty.
 2. Start from First.
 3. Print the data of each node.
 4. Move to the next node.
 5. Stop when First is reached again.
-

DeleteFirst()

1. Check whether the list is empty.
 2. If only one node exists:
 - Delete it.
 - Set First to NULL.
 3. Otherwise:
 - Traverse to the last node.
 - Move First to the next node.
 - Update last node's next.
 - Delete the old first node.
-

DeleteLast()

1. Check whether the list is empty.
 2. If only one node exists:
 - Delete it.
 - Set First to NULL.
 3. Otherwise:
 - Traverse to the second last node.
 - Delete the last node.
 - Update second last node's next to First.
-

AddPosition()

1. Validate the position.
 2. If position is 1, call AddFirst().
 3. If position is Count + 1, call AddLast().
 4. Otherwise:
 - Traverse to the previous node.
 - Insert the new node.
-

DeletePosition()

1. Validate the position.
2. If position is 1, call DeleteFirst().
3. If position is Count, call DeleteLast().
4. Otherwise:
 - Traverse to the previous node.
 - Delete the required node.
 - Update links.

Sample Menu

```
=====
Circular Singly Linked List
=====

1. Add First
2. Add Last
3. Display
4. Delete First
5. Delete Last
6. Add Position
7. Delete Position
8. Count
0. Exit

=====
Enter your choice:
```

Test Cases

Test Case 1

Input

```
AddFirst(30)
```

Output

```
30
```

Test Case 2

Input

```
AddFirst(20)  
AddFirst(10)
```

Output

```
10 20 30
```

Test Case 3

Input

```
AddLast(40)
```

Output

```
10 20 30 40
```

Test Case 4

Input

```
DeleteFirst()
```

Output

```
20 30 40
```

Test Case 5

Input

```
DeleteLast()
```

Output

```
20 30
```

Test Case 6

Input

```
AddPosition(2, 25)
```

Output

```
20 25 30
```

Test Case 7

Input

```
DeletePosition(2)
```

Output

```
20 30
```

Test Case 8

Input

```
DeleteFirst()  
DeleteFirst()  
DeleteFirst()
```


Output

List is Empty

SUNBEAM